

Database Design

The shared schema every backend module reads and writes.

Read this if: backend (M1-M3); frontend can skip unless debugging a data shape

One PostgreSQL database via Supabase, with pgvector for embeddings. There's exactly one shared database — no spinning up a separate one per module.

Core tables

users — owned by Supabase Auth; every app table references this user id.

repositories:	id, user_id, github_url, name, owner, default_branch, status, created_at, updated_at
analysis_jobs:	id, repository_id, status, error_message, started_at, completed_at, created_at
repository_files:	id, repository_id, path, language, content, size, hash, created_at
commits:	id, repository_id, commit_hash, author, message, timestamp, diff
pull_requests:	id, repository_id, external_id, title, body, author, state, created_at, merged_at
issues:	id, repository_id, external_id, title, body, author, state, created_at
contributors:	id, repository_id, external_id, name, email_hash, commit_count, first_seen, last_seen
code_entities:	id, repository_id, file_id, entity_type, name, start_line, end_line, signature
dependencies:	id, repository_id, source_entity_id, target_entity_id, dependency_type
risk_records:	id, repository_id, file_id, score, signals, explanation, created_at
knowledge_items:	id, repository_id, user_id, title, content, source, created_at, updated_at
knowledge_gaps:	id, repository_id, question, context, priority, status, resolved_knowledge_item_id, created_at, resolved_at
knowledge_embeddings:	id, repository_id, knowledge_item_id, content, embedding (pgvector), created_at
evidence:	id, repository_id, source_type, source_id, file_id, commit_id, pull_request_id, issue_id, knowledge_item_id, excerpt, metadata, created_at
offboarding_sessions:	id, repository_id, contributor_id, status, created_at, completed_at
offboarding_questions:	id, session_id, question, priority, answer, status

`evidence.knowledge_item_id` is set when `source_type` is `knowledge_item`, so a human answer can be cited as evidence just like a file, commit, PR, or issue.

Answering a knowledge gap creates a row in `knowledge_items` and records its id in `knowledge_gaps.resolved_knowledge_item_id`, keeping the question linked to what resolved it.

Relationships

User → Repository → {Analysis Job, Files, Commits, Pull Requests, Issues, Contributors, Code Entities, Dependencies, Risks, Knowledge, Knowledge Gaps, Evidence, Offboarding Sessions}
--

Isolation rule

Every repository-scoped query verifies ownership through `user_id`. Never trust a repository id from the client — check it, every time.

Vector retrieval

pgvector is the single retrieval mechanism for human knowledge, repository context, and evidence. No parallel store (Chroma, SQLite, etc.) alongside it.

Schema ownership

Member 1 coordinates schema changes. Anyone can propose a change; nobody stands up a competing schema solo.